ISIL

Informe de desarrollo
Proceso de Aprendizaje 3

Módulo de consulta y supervisión del Sistema de Pedidos

Curso: Desarrollo de Aplicaciones Empresariales Avanzado

NRC: 4622

Estudiante: Adrian Antonio Valdez Lozada

Docente: [Completar con el nombre indicado en el curso]

Periodo: 2026-20

Repositorio GitHub: https://github.com/Adri123Vldz/isil-2026-ii/tree/feature/pa3-adrian

Entrega individual · PA3

1. Objetivo y solución desarrollada

El objetivo de esta PA3 es ampliar el Sistema de Pedidos con funciones de consulta y supervisión, aplicando Spring MVC con JSP, una API REST/JSON, controles de Spring Security y consultas explícitas con Spring JDBC. Se conserva el CRUD existente con Spring Data JPA y la integración HTTP con el servicio de inventario.

El alcance funcional de esta iteración no vuelve a implementar las operaciones completas de edición/eliminación ni el flujo de anulación de PA2. Se concentra en consultar los pedidos almacenados y consolidar indicadores por cliente.

Arquitectura de la solicitud

```
Navegador → Spring Security → DispatcherServlet → Controller JSP / API JSON →
PedidoReporteService → PedidoReporteJdbcRepository → MySQL
PedidoService → PedidoRepository (JPA) para pedidos · InventarioRestClient → inventario-service
para operaciones de inventario
```

La vista JSP y los dos nuevos endpoints JSON llaman al mismo servicio de reportes. La búsqueda avanzada y los agregados se ejecutan contra el historial local; no consultan inventario-service. El diagrama completo de dos niveles se adjunta como archivo Markdown en el proyecto.

Organización por capa

Capa	Responsabilidad PA3	Componentes representativos
Presentación	Recibir solicitudes, entregar JSP o JSON.	PedidoReporteWebController, PedidoReporteController y reportes-pedidos.jsp.
Negocio	Validar parámetros y compartir operaciones de reporte.	PedidoReporteService e implementación.
Persistencia	SQL parametrizado y mapeo de resultados; JPA permanece para CRUD.	PedidoReporteJdbcRepository, PedidoRowMapper y proyección agregada.
Seguridad	Autenticación, autorización por rol y protección CSRF.	SecurityConfig y filtros de Spring Security.

La implementación está publicada en el repositorio personal, en la rama indicada en la carátula.

2. Consultas con Spring JDBC

Búsqueda avanzada

`buscarPedidosPorClienteYTotalMinimo(cliente, totalMinimo)` utiliza `NamedParameterJdbcTemplate` con parámetros separados para el cliente y el monto. La búsqueda parcial no distingue mayúsculas/minúsculas, filtra `total >= totalMinimo`, reutiliza `PedidoRowMapper` y ordena por ID descendente.

Clientes frecuentes

`listarClientesFrecuentes(minPedidos)` agrupa por cliente y aplica `HAVING COUNT(*) >= :minPedidos`. La proyección retorna:

- **totalPedidos:** cantidad de pedidos incluidos.
- **totalUnidades:** suma de las cantidades.
- **importeAcumulado:** suma del total de pedidos.
- **ticketPromedio:** promedio del total por pedido.

Los valores aportados por el usuario no se concatenan al SQL. La capa Service exige un cliente no vacío, un monto mínimo no negativo y un mínimo de pedidos igual o mayor a uno. Las entradas inválidas se responden como HTTP 400 en la API.

Convivencia de persistencia

Mecanismo	Uso	Base de datos
Spring Data JPA	Operación CRUD transaccional existente de pedidos.	MySQL en ejecución normal; H2 en pruebas y demostración local.
Spring JDBC	Reportes de solo lectura y agregados explícitos requeridos por PA3.	El mismo DataSource MySQL; H2 en pruebas y demostración local.
HTTP RestClient	Consulta/reserva de inventario durante el alta de pedidos.	Servicio Jakarta EE externo con SQL Server.

Separar las consultas de reporte de la persistencia JPA permite cumplir el requisito de SQL explícito sin sustituir el flujo existente. H2 se activa en pruebas y mediante el perfil local `demo`, que carga cinco pedidos deterministas. Este perfil permite demostrar las consultas sin MySQL, pero no sustituye la validación con MySQL.

3. Spring MVC, API y seguridad

Vista web

`GET /reportes/pedidos` se atiende mediante un controlador Spring MVC que recibe `Model` y selecciona la vista `reportes-pedidos.jsp`. La página presenta el resumen de pedidos, un formulario de cliente y monto mínimo, y un formulario para indicar el umbral de frecuencia. No genera HTML desde el controlador ni duplica reglas de negocio.

Endpoints JSON

Método y ruta	Resultado
GET /api/v1/pedidos/busqueda-avanzada?cliente=Ana&totalMinimo=100	Arreglo JSON de pedidos que cumplen ambos filtros.
GET /api/v1/pedidos/clientes-frecuentes?minPedidos=2	Arreglo JSON con los clientes que alcanzan el umbral y sus cuatro métricas.

Controles de acceso

- Los roles `OPERADOR` y `CONSULTA` pueden realizar GET a los reportes web y REST.
- El rol `CONSULTA` no puede registrar pedidos con POST a `/pedidos`.
- Un usuario sin sesión es dirigido al formulario de autenticación.
- CSRF permanece habilitado. El POST requiere token válido incluso para `OPERADOR`.

La autenticación es de laboratorio, con usuarios en memoria y claves externalizables. No se agregan JWT, OAuth2 ni persistencia de usuarios, porque no forman parte del requerimiento.

Continuidad ante inventario no disponible

Las consultas nuevas se apoyan en el historial local, por lo que su ruta de ejecución no requiere el cliente HTTP de inventario. La operación de alta sí depende de inventario-service para consultar o reservar el stock. La prueba automatizada confirma que el alta falla sin persistir cuando se rechaza la conexión HTTP y que el reporte local sigue disponible. Además, el 31/10/2026 el catálogo de inventario respondió HTTP 200 con tres productos y las consultas de stock devolvieron 10, 20 y 30; fueron solo solicitudes GET, sin reservar ni alterar existencias.

4. Pruebas automatizadas y casos funcionales

Resultado automatizado verificado

Se ejecutó `mvnw clean test` en el proyecto Spring Boot. Resultado verificado: **38 pruebas, 0 fallos, 0 errores, 0 omitidas**; 18 son nuevas para PA3 respecto de la rama base del docente. El registro de consola y los reportes Surefire están en `entrega-pa3/evidencias/automatizadas/`.

Prueba	Verificación añadida para PA3	Resultado
Repositorio JDBC	Filtro cliente/monto, orden descendente y agregados GROUP BY/HAVING.	Correcta con H2
Servicio de reportes	Rechazo de cliente vacío, monto negativo y umbral menor a uno.	Correcta con Mockito
API y MVC	JSON de ambos reportes, HTTP 400 para cliente vacío y valores inválidos, y filtros de cliente/monto y frecuencia en el modelo JSP.	Correcta con MockMvc/H2
Perfil demo H2	Contexto Spring arranca con los cinco pedidos semilla; búsqueda avanzada y agregados verifican los valores esperados.	Correcta con Spring Boot/H2
Inventario no disponible	Conexión HTTP rechazada: el alta falla sin persistir y el reporte local continúa disponible.	Correcta con H2 e integración HTTP local
Seguridad	Acceso de CONSULTA, redirección de anónimo, rechazo de POST no autorizado y protección CSRF.	Correcta con Spring Security Test

Estado de los cinco escenarios

> **Suite automatizada aprobada; recorrido manual con MySQL pendiente:** MySQL no está instalado y el puerto 3306 no acepta conexiones; tampoco quedó confirmado Sistema de Pedidos escuchando en el puerto 8081. El servicio de inventario sí respondió en el puerto 8082 durante una comprobación GET separada. No se afirma haber ejecutado los cinco escenarios manualmente ni contra MySQL, y no se incluyen capturas ficticias.

1. Búsqueda por cliente y monto: repositorio, API y modelo JSP se verifican con H2/MockMvc; el perfil demo confirma datos reproducibles. Falta el recorrido HTTP/manual y contraste con MySQL.
2. Clientes frecuentes: las cuatro métricas se verifican con H2; el perfil demo confirma 3 pedidos, 8 unidades, S/ 5,580 y ticket promedio S/ 1,860 para Ana Torres. Falta el contraste con MySQL.
3. Parámetros inválidos: cliente vacío, monto negativo y umbral menor a uno producen HTTP 400 en pruebas automatizadas; falta evidencia HTTP/manual.
4. Inventario indisponible: una prueba de integración confirma que el alta falla sin guardar y que el reporte local sigue disponible; el inventario real se consultó por separado mediante GET. Falta repetir el escenario coordinado con la aplicación y MySQL.
5. Roles y CSRF: MockMvc confirma autorización de CONSULTA, redirección de anónimo, rechazo de POST no autorizado y CSRF requerido; falta evidencia HTTP/manual.

La matriz y los pasos reproducibles están en `entrega-pa3/evidencias/README.md`. Las pruebas automatizadas con H2 no sustituyen las capturas manuales requeridas con MySQL y el servicio externo.

5. Evidencia visual de los reportes

La vista JSP presenta las métricas generales en dos columnas: conceptos a la izquierda y valores alineados a la derecha. Las capturas siguientes se obtuvieron desde la aplicación local con el usuario de consulta y el perfil demo de H2.

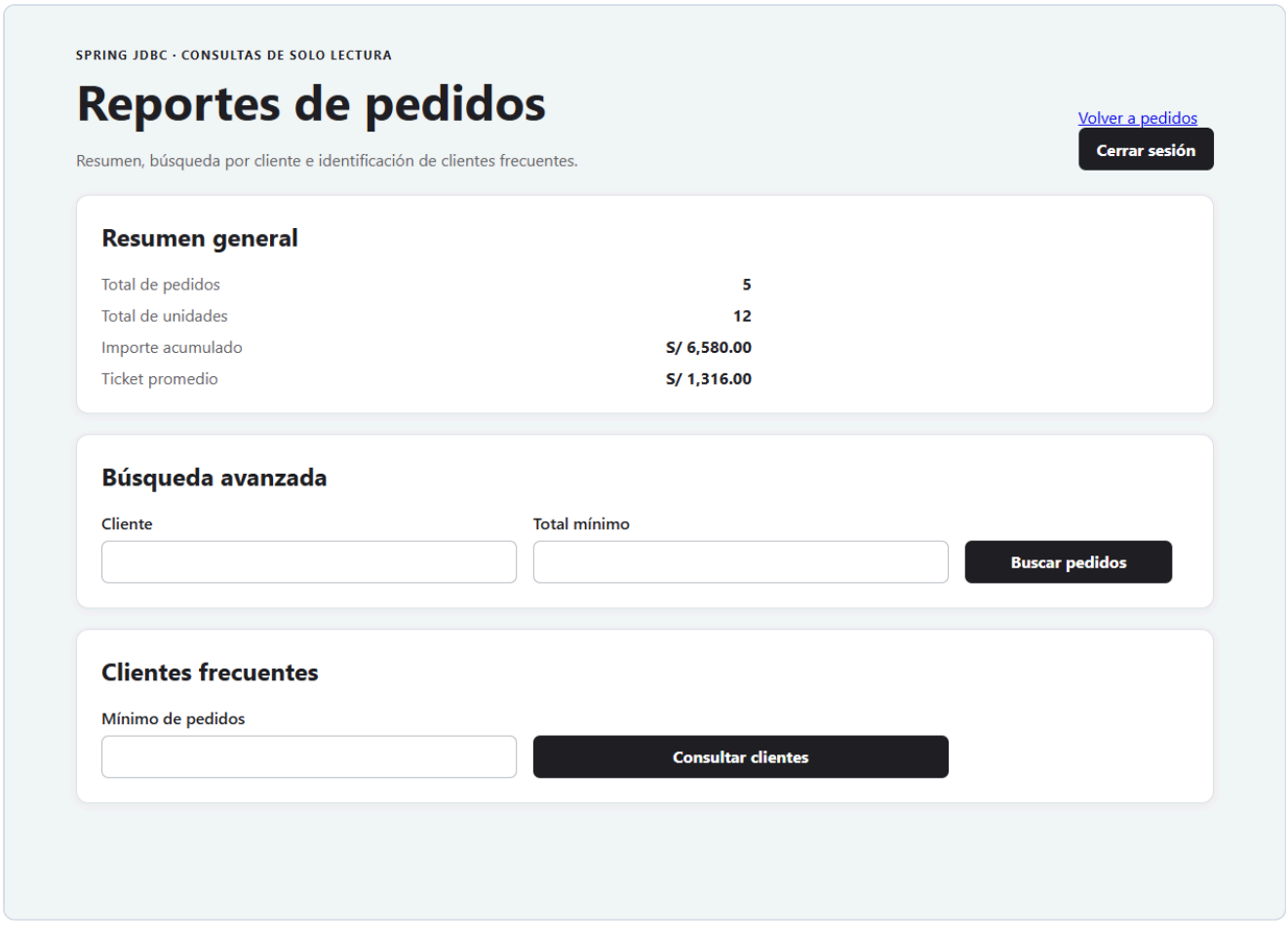


Figura 1. Vista de reportes y diseño actualizado del resumen general. Datos de demostración en H2.

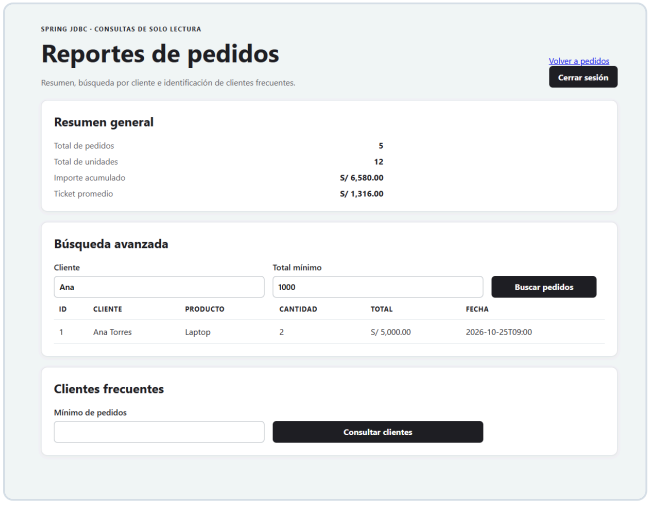


Figura 2. Búsqueda por cliente Ana y total mínimo S/ 1,000: pedido de Laptop por S/ 5,000.

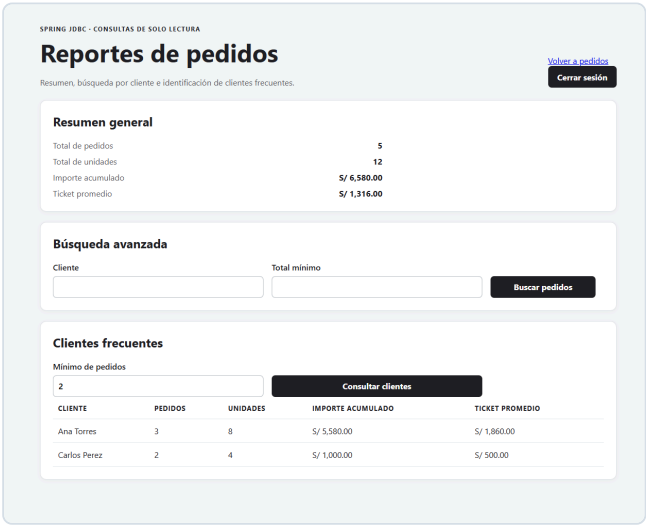


Figura 3. Clientes frecuentes con umbral de dos pedidos y sus cuatro métricas agregadas.

Estas capturas documentan una demostración web real con H2 y datos semilla; no son una validación de MySQL ni reemplazan las evidencias pendientes de los cinco escenarios manuales.

6. Conclusiones y referencias

Conclusiones

1. La aplicación conserva la separación entre controladores, servicio y repositorio, comparte la lógica de reportes entre JSP y API y presenta el resumen con etiquetas y valores alineados en dos columnas.
2. Las dos consultas nuevas utilizan Spring JDBC con parámetros nombrados; el reporte agregado usa GROUP BY/HAVING y una proyección tipada.
3. La configuración conserva la autorización por rol y CSRF. La suite automatizada confirma el comportamiento cubierto bajo H2.
4. La rama de trabajo está publicada en el fork personal y las 38 pruebas automatizadas pasan. El perfil demo de H2 facilita una ejecución reproducible sin MySQL; para acreditar la compatibilidad solicitada y cerrar la evidencia manual todavía se requiere ejecutar los cinco escenarios con MySQL y los servicios activos.

Referencias

Spring. (s. f.). *JDBC*. Spring Framework Reference Documentation. https://docs.spring.io/spring-framework/reference/data-access/jdbc.html

Spring. (s. f.). *Web MVC*. Spring Framework Reference Documentation. https://docs.spring.io/spring-framework/reference/web/webmvc.html

Spring. (s. f.). *Authorize HTTP requests*. Spring Security Reference. https://docs.spring.io/spring-security/reference/servlet/authorization/authorize-http-requests.html

Spring. (s. f.). *Cross Site Request Forgery (CSRF)*. Spring Security Reference. https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html

Spring. (s. f.). *Testing*. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/reference/testing/index.html

Archivos de apoyo

- Diagrama: `.github/modernize/assessment/engines/facts/architecture-diagram.md`.
- README, instrucciones de configuración y pruebas: `README.md`.
- Matriz de evidencia y registro automatizado: `entrega-pa3/evidencias/`.

La carátula conserva el campo Docente tal como fue solicitado. Adjuntar la evidencia manual real de los cinco escenarios cuando se ejecute en el entorno con MySQL.